

Introdução à Programação



Conceitos Básicos de Orientação a Objetos

O que é Orientação a Objetos?

- ◆ É considerar que tudo é um objeto:
 - Os sistemas são objetos
 - Os dados manipulados e comunicados entre os sistemas são objetos
 - Cada parte de um sistema (sub-sistema) é um objeto
 - Os dispositivos eletrônicos são objetos
 - A interface com o usuário é um objeto, composto de vários outros objetos...

Objeto

- ◆ Podemos definir **objeto** como uma entidade que possui:
 - **Estado**
 - Características ou propriedades do objeto
 - **Comportamento**
 - Operações que o objeto pode realizar
- ◆ O comportamento pode examinar ou alterar o estado de um objeto

Objeto DVD

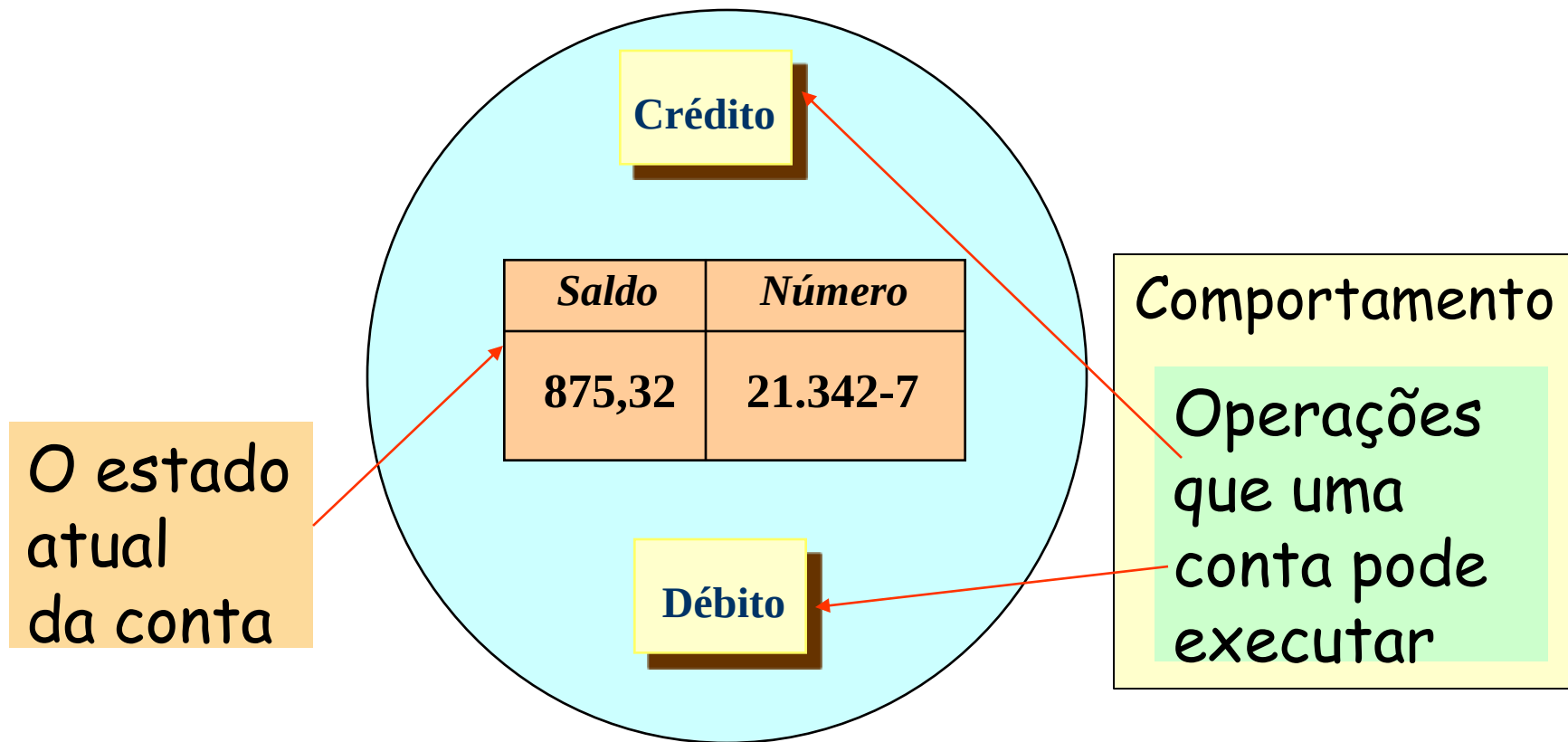
As operações que o DVD pode executar



O estado atual do DVD; o que ele está fazendo...

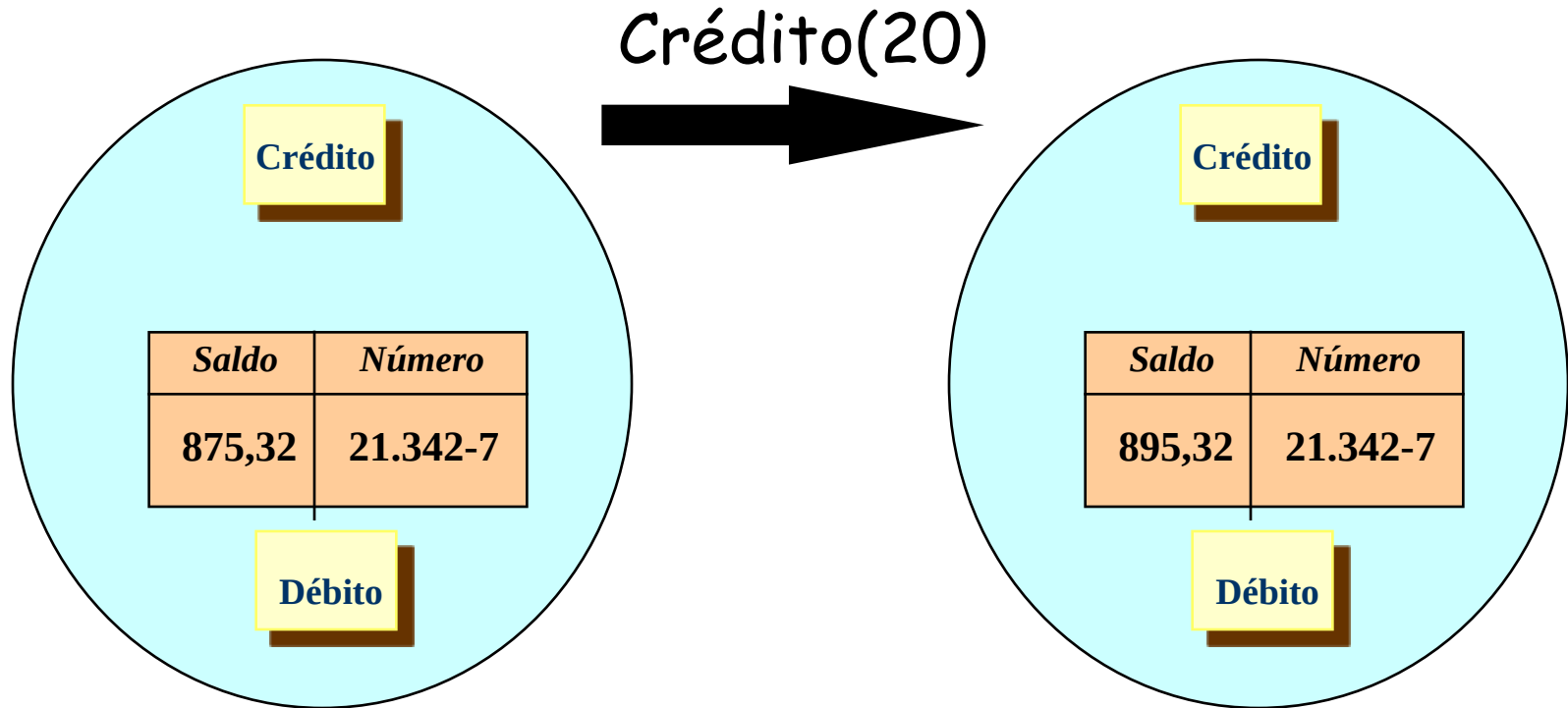
Fonte: <http://www.amazon.com>

Objeto Conta Bancária



Estados do Objeto Conta

- ❖ Comportamento neste caso mudou o estado do objeto conta

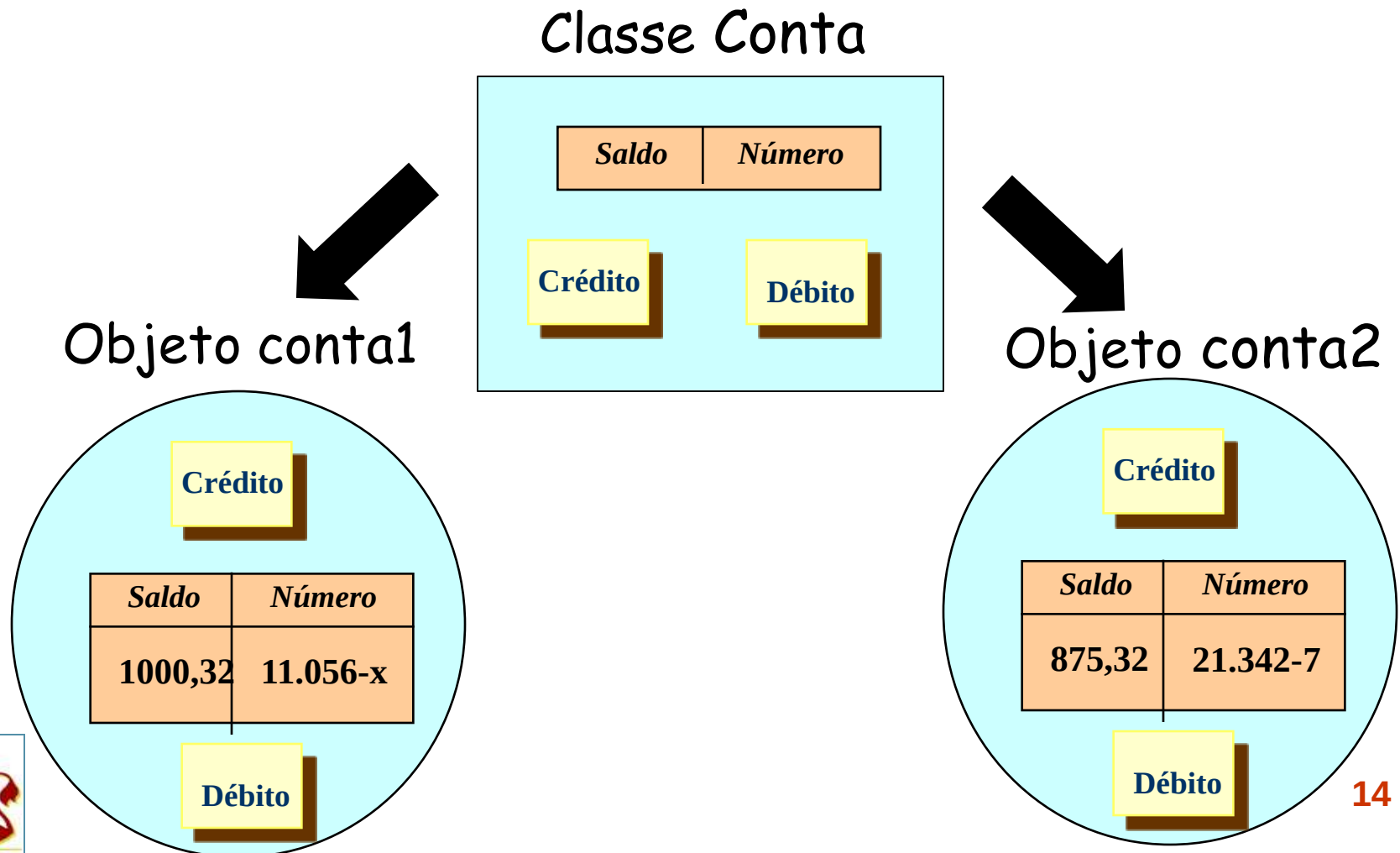


Classe

- ❖ **Classe** é um agrupamento de objetos que têm propriedades comuns e podem realizar as mesmas operações
- ❖ É uma definição que descreve como objetos pertencentes à classe são estruturados internamente (quais propriedades e operações o objeto possui)
- ❖ Classe é um conceito e o objeto é uma instância deste conceito
- ❖ Portanto, podemos ter vários objetos pertencentes a mesma classe

Classe x Objeto

- ◆ Múltiplos objetos podem ser criados à partir da mesma classe



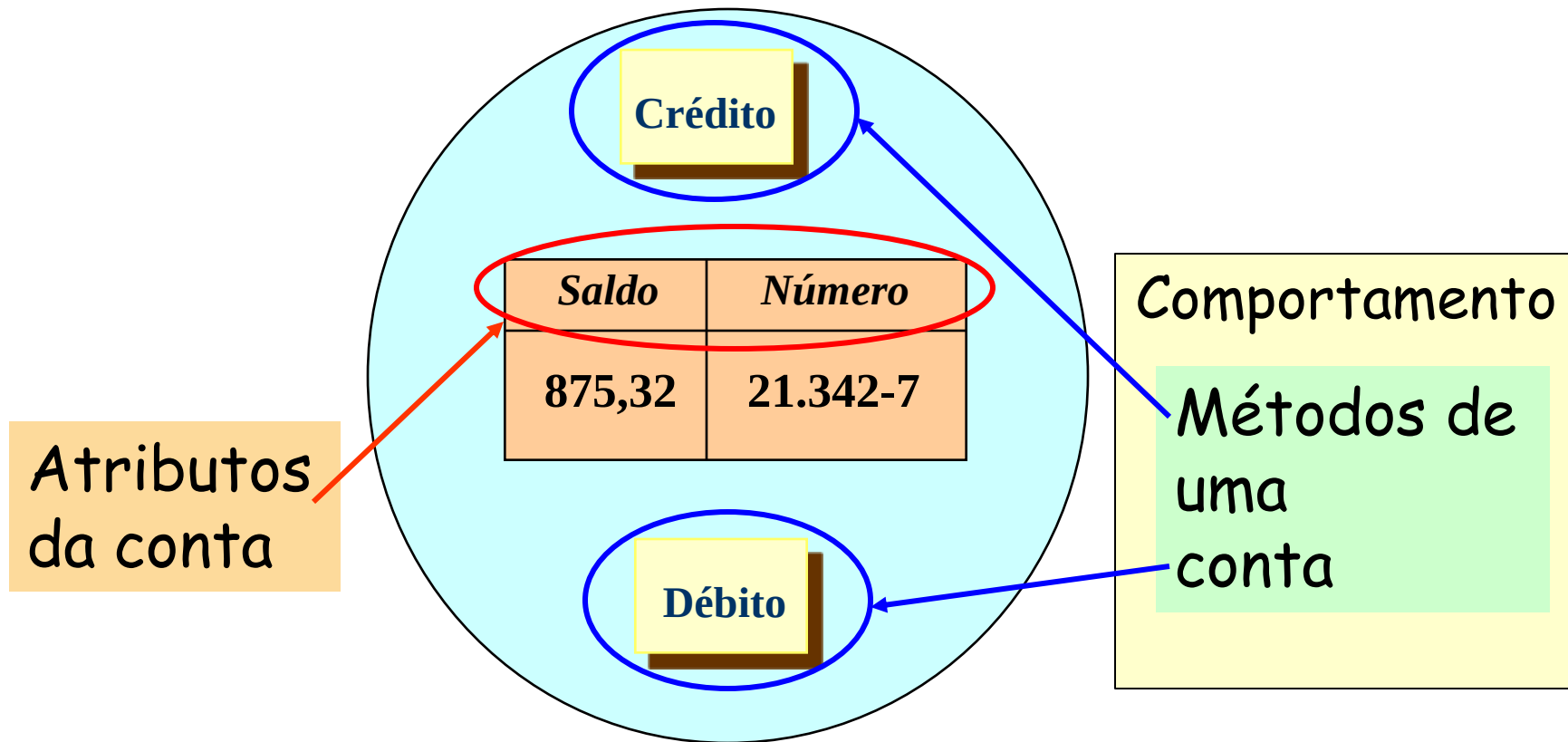
Classe e Tipo

- ◆ Tipo é um conjunto de valores relacionados que são capazes de realizar as mesmas operações
 - Ex: inteiro, caractere, Conta, Banco, etc
- ◆ Uma classe define um **tipo**: o tipo cujos elementos são objetos com as mesmas propriedades e comportamento

Atributo e Método (1)

- ❖ **Atributo** representa alguma propriedade do objeto
- ❖ O conjunto de atributos de um objeto determina o estado do objeto
- ❖ Um atributo pode conter um valor simples tal como um número real ou até outro objeto
- ❖ **Método** representa uma operação que um objeto pode fazer
- ❖ O conjunto de métodos de um objeto determina o comportamento do objeto

Atributo e Método (2)

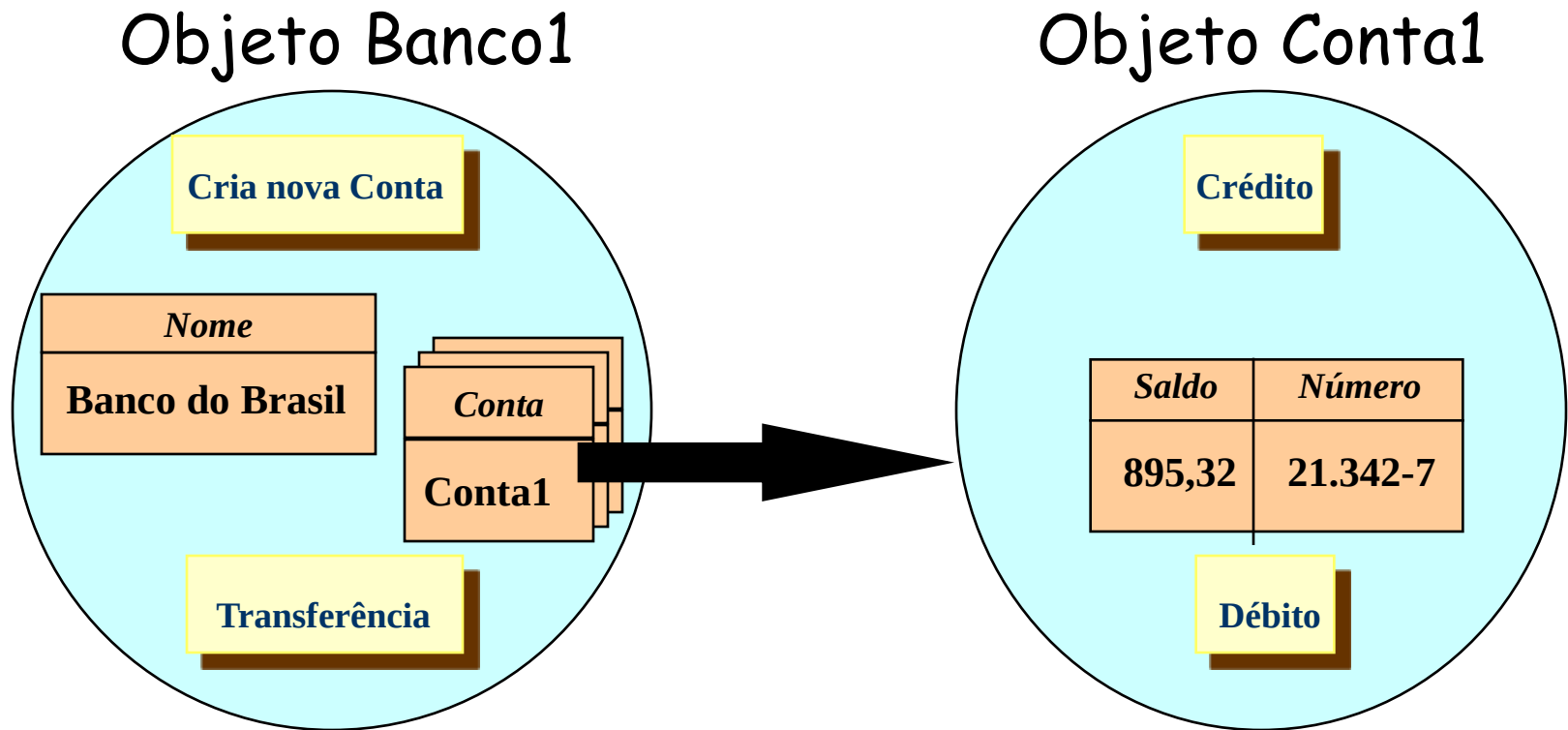


Relações entre Objetos

- ◆ Geralmente, um software é construído a partir de vários objetos (classes) que se relacionam
- ◆ As relações de **Cliente** e **Herança** são essenciais para a construção de programas
- ◆ Classe *B* é **Cliente** de classe *A*, se todo objeto de tipo *B* contiver informação sobre um ou mais objetos do tipo *A*
- ◆ Classe *B* é **Herdeira** de classe *A*, se *B* representa uma versão especializada de *A*
 - Métodos e os atributos da classe *A* se tornam, também, parte da classe *B*
 - Características e comportamento de *A* são passadas para *B*

Relação Cliente

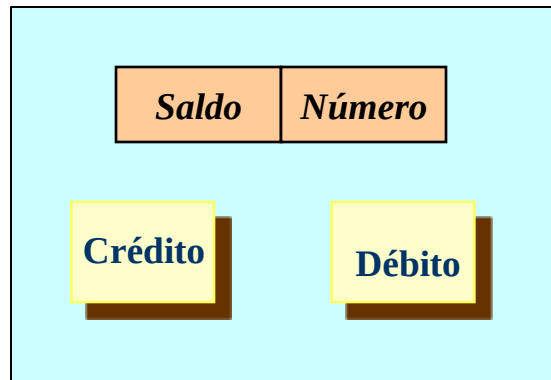
- Objeto do tipo Banco têm vários objetos do tipo Conta ➡ Banco é **cliente** de Conta



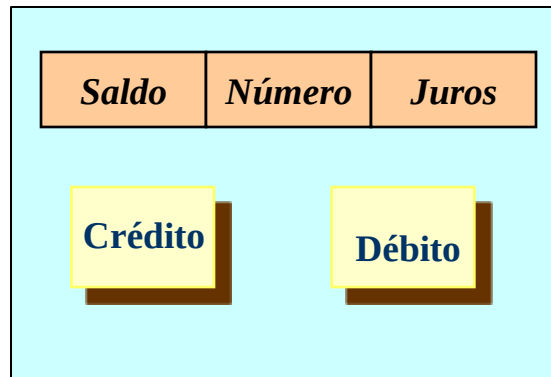
Relação Herança

- ◆ Classe Poupança é uma versão especializada de Conta
 - ➔ Métodos e atributos são **herdados**

Classe Conta



Classe Poupança



Utilizando OO para Desenvolver Programas

- ◆ Desenvolvimento de uma solução se torna mais fácil quando quebramos esta solução em módulos gerenciáveis
- ◆ Quando estamos programando, podemos desenvolver módulos separados, onde cada um é responsável por uma certa parte da solução
 - Programação Modular
- ◆ OO facilita a programação modular
 - Módulos são as classes e objetos



Algumas Considerações sobre OO

- ◆ Orientação a Objetos ➡ Modularidade
 - Reusabilidade
 - Extensibilidade
- ◆ Classe é uma entidade que combina 2 conceitos:
Módulo e Tipo
 - **Módulo** é um conceito **sintático**, pois a decomposição do sistema em módulos afeta **somente** a forma como o programa é escrito, não a funcionalidade dele
 - **Tipo** é um conceito **semântico**, pois o tipo influencia diretamente a execução do sistema, definindo a forma dos objetos que são criados e manipulados pelo sistema em tempo de execução

OO e Linguagens de Programação

- ◆ Linguagens OO têm em objetos e classes seus elementos fundamentais para construção de programas
 - Estruturas da linguagem permitem mapeamento direto dos conceitos de OO
- ◆ Porém, é possível fazer um programa OO em uma linguagem não OO
 - É possível, também, fazer um programa que não seja OO em uma linguagem OO

Conceito de Orientação a Objetos é independente da linguagem de programação

Linguagem C++

- ◆ Criada por Bjarne Stroustrup em 1983
- ◆ Pode ser vista **quase** como uma extensão da linguagem C para dar suporte a OO
 - Embora quase tudo escrito em C seja compatível com C++, certos comandos podem ter comportamento diferente em C++
- ◆ Dá suporte a vários conceitos de OO:
 - Classe, Herança, Polimorfismo, Encapsulamento
- ◆ Ainda oferece suporte a tratamento de exceções e tipos genéricos
- ◆ Sintaxe da linguagem Java foi inspirada em C++

Um Programa Imperativo...

Contém

- Uma função principal, por onde começa a execução do programa
- Várias funções auxiliares, para modularizar, dividir o código em partes
- Importação de bibliotecas de funções

◆ Pode ainda conter:

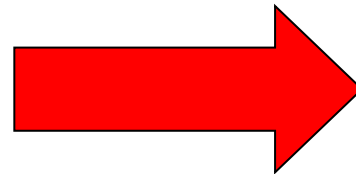
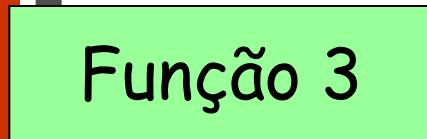
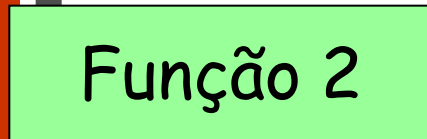
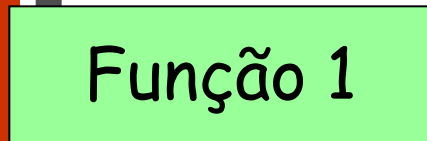
- Definição de tipos estruturados

Um Programa Orientado a Objetos ...

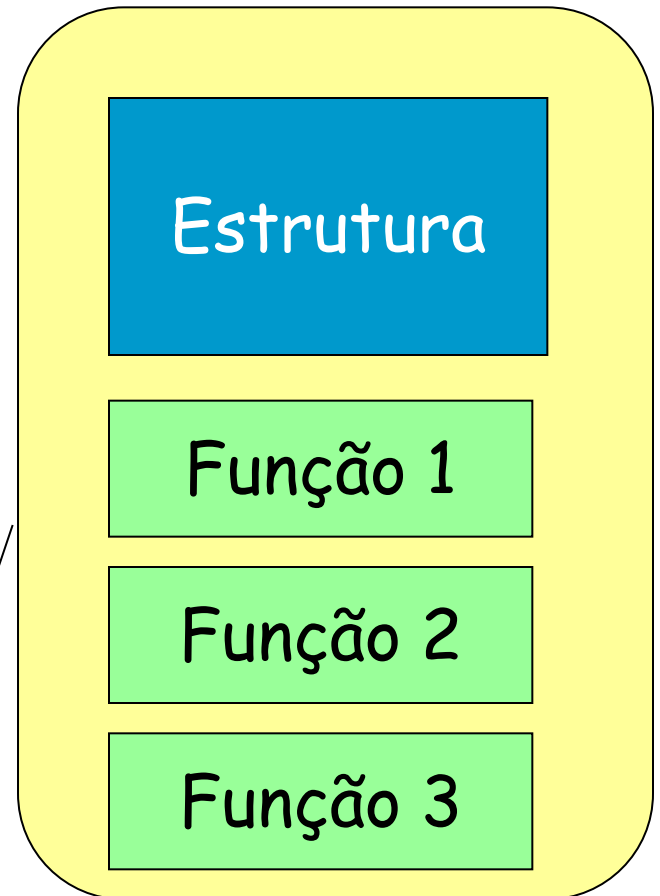
- ◆ Tem a **classe** como a sua construção fundamental :
 - Um programa é constituído de uma ou mais classes
 - Que podem ter sido importadas de uma biblioteca de classes
 - Uma classe contém um ou mais atributos e métodos
 - Um método é composto de instruções

Ao Invés de Estruturas, Classes!

Classe



Define um tipo,
com funções que
operam sobre
este tipo



Programação Imperativa x Programação OO

- ◆ Programação Imperativa tem como elementos principais os procedimentos (funções)
- ◆ Programação OO tem como elementos principais as classes
 - Estrutura + funções = classe
- ◆ Programação OO ainda utiliza :
 - Herança
 - Encapsulamento
 - Polimorfismo

Veremos depois!

Implementando o Tipo Conta com Programação Imperativa

```
struct Conta{  
    double saldo;  
    char numero[11];  
    char agencia[11];  
};
```

- ◆ Primeiro passo é definir um tipo estruturado Conta

Implementando as Funções do Tipo Conta

Passa-se estrutura (ou endereço de estrutura) como parâmetro

```
void creditar(struct Conta *c, double v) {  
    c->saldo = c->saldo + v;  
}
```

◆ Depois se define as funções que operam sobre o tipo Conta

- creditar
- debitar
- getSaldo
- criaConta

Implementando as Funções do Tipo Conta

```
void debitar(struct Conta *c, double v) {  
    if (v <= c->saldo)  
        c->saldo = c->saldo - v;  
}
```

```
double getSaldo(struct Conta *c) {  
    return c->saldo;  
}
```

Implementando as Funções do Tipo Conta

```
struct Conta* criaConta(char *num, char* ag) {  
    struct Conta *retorno;  
    retorno = (struct Conta *)  
              malloc (sizeof(struct Conta));  
    if (retorno == NULL){  
        printf("Erro na alocação de memória!")  
    } else {  
        strcpy(retorno->numero, num);  
        strcpy(retorno->agencia, ag);  
        retorno->saldo = 0;  
    }  
    return retorno;  
}
```

Alocando dinamicamente uma
estrutura

Usando o Tipo Conta em Programação Imperativa

```
int main() {  
    struct Conta *conta1;  
    conta1 = criaConta("1234-5", "007-x");  
    if (conta1 == NULL) {  
        printf("Erro na alocação de memória!")  
    } else {  
        creditar(conta1, 300);  
        printf("Saldo da conta = %lf\n", getSaldo(conta1));  
        debitar(conta1, 200);  
        printf("Saldo da conta = %lf\n", getSaldo(conta1));  
    }  
    return 0;  
}
```

Implementando o Tipo Conta com Programação OO (C++)

```
class Conta{  
    private:
```

Atributos

```
        double saldo;  
        char numero[11];  
        char agencia[11];
```

Métodos

```
    public:
```

```
        void creditar(double v);  
        void debitar(double v);  
        double getSaldo();  
        Conta(char* num, char* ag);
```

```
};
```

Construtor

◆ Primeiro passo é definir a classe Conta

● Atributos e Métodos

Implementando os Métodos do Tipo Conta

Indica que método faz parte da classe Conta

```
void Conta::creditar(double v) {  
    saldo = saldo + v;  
}
```

saldo é um atributo que foi definido na classe Conta

Não precisa passar objeto como parâmetro!

Implementando os Métodos do Tipo Conta

```
void Conta::debitar(double v) {  
    if (v <= saldo)  
        saldo = saldo - v;  
}
```

```
double Conta::getSaldo() {  
    return saldo;  
}
```